A Mini Project Report

on

Mitigating Context Degradation in LLM's Using Hybrid Retrieval and External Memory

in

COMPUTER SCIENCE AND ENGINEERING

by

Gubbala Mani Varsha (160123733301)
Polabathina Ramcharan Teja (160123733303)

Department of Computer Science and Engineering,
Chaitanya Bharathi Institute of Technology (Autonomous),
(Affiliated to Osmania University, Hyderabad)
Hyderabad, TELANGANA (INDIA) –500 075
[2025-2026]

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute I Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in
Approved by | Affiliated to | UGC Autonomous | 12 Programs Accredited by NBA | Grade A++ in NAAC | All India Ranking 151-200 Band nirf | Certified by ISO 21001 : 2018
COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 47 years

CERTIFICATE

This is to certify that the mini project titled "**Mitigating Context Degradation in LLM's Using Hybrid Retrieval and External Memory**" is the Bonafide work carried out by **G. Mani Varsha and P. Ramcharan Teja** , students of B.E.(CSE) of Chaitanya Bharathi Institute of Technology(A), Hyderabad, affiliated to Osmania University, Hyderabad, Telangana(India) during the academic year 2025-2026, submitted in partial fulfillment of the requirements for the VI semester degree in **Bachelor of Engineering (Computer Science and Engineering)** and that this work has not formed the basis for the award previously of any other degree, diploma, fellowship or any other similar title.

Mini Project Coordinator
Asst. Prof. A. Mohan

Head, CSE Dept.
Dr. S. China Ramu

Place: Chaitanya Bharathi Institute of Technology (Gandipet, Hyderabad)
Date:

DECLARATION

I/we hereby declare that the mini project entitled "**Mitigating Context Degradation in LLM's Using Hybrid Search and External Memory**" submitted for the VI Semester B.E (CSE) degree is our original work, and the mini project has not formed the basis for the award of any other degree, diploma, fellowship or any other similar titles.

Name(s) and Signature(s) of the Student

Place: Chaitanya Bharathi Institute of Technology (Gandipet, Hyderabad)
Date:

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to all those who provided us the possibility to complete this mini project. A special gratitude we give to our project guide, A. Mohan, whose contribution in stimulating suggestions and encouragement helped us to coordinate our project especially in writing this report.
We also acknowledge with a deep sense of reverence, our gratitude towards the Head of the Department, CSE, Dr. S. China Ramu, for providing us with the necessary facilities and support.

ABSTRACT

Transformer-based Large Language Models (LLMs) frequently experience ”attention dilution” and the ”lost-in-the-middle” effect when processing extensive context windows, leading to severe computational overhead and decreased factual retrieval accuracy. This mini project presents a highly optimized, modular Retrieval-Augmented Generation (RAG) system engineered to mitigate context degradation. By decoupling long-form document storage from LLM contextual inference, the system enforces a strict query-time token budget. A dual-index methodology was implemented utilizing FAISS for dense semantic vector searches and BM25 for precise lexical keyword matching. These concurrent retrieval streams are unified via Reciprocal Rank Fusion (RRF) and optimized using a cross-encoder reranked to extract only the highest-scoring text chunks. Furthermore, the architecture integrates query-level and response-level caching mechanisms to drastically reduce API latency and redundant computational costs. As a practical deployment, the system was extended into a cross-platform browser extension acting as a conversational memory layer for platforms like ChatGPT and Gemini. Evaluation demonstrates a 60% reduction in token usage while significantly stabilizing central-context retrieval accuracy.

Table of Contents

List of Figures

List of Tables

CHAPTER 1
INTRODUCTION

Large Language Models have emerged as a foundational technology in modern artificial intelligence, enabling significant advancements in natural language understanding, generation, and reasoning. Models based on the Transformer architecture have demonstrated strong performance across a wide range of tasks, including question answering, summarization, and conversational systems. However, despite these capabilities, their effectiveness is significantly challenged when dealing with long-context inputs. As the length of input sequences increases, issues such as attention dilution, positional bias, and computational inefficiency become more prominent, leading to degradation in performance. These limitations highlight the need for more efficient mechanisms that can selectively focus on relevant information without processing entire documents. In this context, retrieval-augmented approaches provide a promising direction by combining external memory with language models to improve both efficiency and accuracy

1.1 Problem Statement

Large Language Models (LLMs), particularly those based on the Transformer architecture, have achieved significant success in various Natural Language Processing tasks such as question answering, summarization, and conversational systems. However, their performance degrades when handling long input sequences.

This degradation is primarily caused by three key limitations: attention dilution, positional bias (known as the lost-in-the-middle effect), and high computational complexity.

First, attention dilution occurs because the self-attention mechanism distributes attention across all tokens in a sequence. As the length of the input increases, the importance assigned to each individual token decreases. Consequently, relevant information may receive insufficient attention, leading to weaker reasoning and reduced accuracy.

Second, LLMs exhibit a positional bias where they tend to focus more on information located at the beginning and the end of a sequence, while often ignoring content present in the middle. This phenomenon, known as the lost-in-the-middle effect, has been experimentally validated in recent

research. It shows that even when important information exists within the input, the model may fail to retrieve it if it is positioned centrally.

Third, the computational cost of the self-attention mechanism grows rapidly with input size. Since each token attends to every other token, the total number of computations increases quadratically with the sequence length. This results in higher memory usage, increased latency, and reduced efficiency when processing long documents.

These limitations make it difficult for LLMs to effectively handle large-scale textual data such as legal documents, research papers, or enterprise knowledge bases. Simply increasing the context window size does not solve the problem, as it further amplifies computational cost and attention dilution.

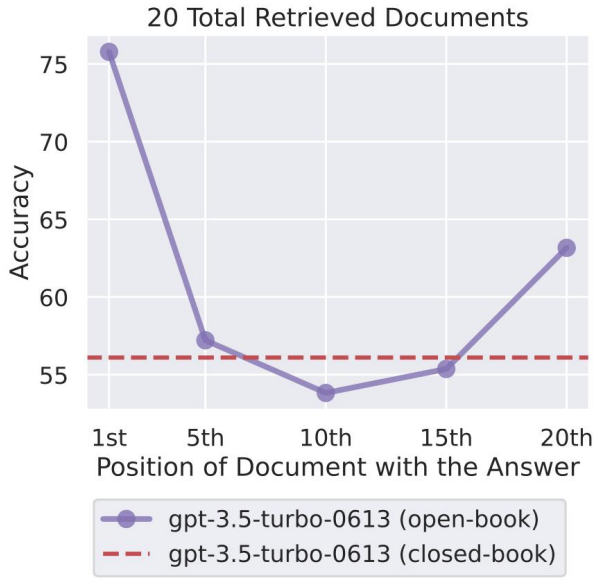

Figure 1.1: Retrieval Accuracy as a Function of Answer Position in the Context Window

Description:

The figure illustrates how retrieval accuracy varies depending on the position of relevant information within a long input sequence. The graph follows a U-shaped pattern, where accuracy is highest at the beginning and end of the sequence, and lowest in the middle. This demonstrates the presence of positional bias in large language models and highlights the limitations of long-context processing.

1.2 Objectives of the Mini Project

The primary objective of this project is to design and implement a system that mitigates context degradation in Large Language Models while improving efficiency and response quality.

The specific objectives are as follows:

- To design a retrieval-augmented architecture that avoids processing entire documents as input
- To implement a hybrid retrieval mechanism combining semantic and lexical search techniques
- To integrate FAISS for semantic similarity search and BM25 for keyword-based retrieval
- To combine retrieval results using Reciprocal Rank Fusion for improved ranking
- To enhance retrieval accuracy using a cross-encoder reranking model
- To enforce a strict token budget in order to reduce computational overhead
- To implement dual-layer caching to reduce latency and redundant computations
- To develop a browser extension that enables cross-platform conversational memory

1.3 Scope of the Mini Project

The scope of this project includes the development of a modular backend system for efficient long-document processing using retrieval-based techniques. The system is designed to support scalable and efficient handling of large textual datasets.

The project also extends to a real-world application in the form of a browser extension that integrates with platforms such as ChatGPT, Gemini, and Claude. This extension allows the system to store and retrieve previous conversational data, thereby enabling persistent memory across sessions.

The proposed system can be applied in various domains, including legal document analysis, medical record processing, financial auditing, enterprise knowledge management, and scientific research.

1.4 Motivation

With the increasing use of Large Language Models in real-world applications, there is a growing need to process long and complex documents efficiently. Traditional approaches rely on increasing the context window size of the model. However, this approach has several limitations.

Increasing the context window leads to higher computational costs and slower response times. It also does not address the underlying issues of attention dilution and positional bias. As a result, the model may still fail to retrieve relevant information effectively, even when it is present within the input.

This motivates the need for an alternative approach that focuses on selecting only the most relevant portions of the input rather than processing the entire document. By using external memory and retrieval-based techniques, it is possible to reduce the input size while maintaining high relevance and accuracy.

The proposed hybrid retrieval architecture addresses these challenges by combining semantic and lexical search methods, enforcing token limits, and optimizing the retrieval process. This results in a more efficient and scalable solution for long-context processing.

1.5 Organization of the Report

The remainder of this report is organized as follows:

- Chapter 2 presents a detailed literature survey covering existing approaches and related work in long-context modeling and retrieval systems
- Chapter 3 describes the design and architecture of the proposed system
- Chapter 4 explains the implementation details, including system modules and methodologies used
- Chapter 5 presents the experimental results, performance analysis, and observations
- Chapter 6 concludes the report and discusses future work and possible improvement.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction to the Problem Domain Terminology

Large Language Models (LLMs) are advanced Natural Language Processing systems built primarily on the Transformer architecture, which relies on self-attention mechanisms to model contextual relationships within text. These models have achieved state-of-the-art performance in tasks such as question answering, summarization, and conversational systems.

However, when processing long input sequences, LLMs exhibit performance degradation due to three major factors: attention dilution, positional bias, and computational complexity. Attention dilution occurs because the model distributes its focus across a large number of tokens, reducing the importance of relevant information. Positional bias, commonly referred to as the "lost-in-the-middle" effect, causes the model to prioritize information at the beginning and end of the input while ignoring content in the middle. Additionally, the self-attention mechanism scales quadratically with input length, leading to increased computational cost and latency.

To address these challenges, retrieval-based approaches have been introduced. These methods allow LLMs to access external knowledge sources and retrieve only the most relevant information, thereby improving efficiency and accuracy.

2.2 Existing Solutions

Several approaches have been proposed to address long-context limitations in LLMs:

2.2.1 Transformer-based Long Context Scaling

Increasing the context window size has been a common approach to improve performance. Models with extended context windows (such as 32K or 128K tokens) attempt to process more information in a single pass. However, studies show that this approach leads to diminishing returns, as attention dilution and computational costs increase significantly [1].

2.2.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, introduced by Lewis et al. [2], enhances LLMs by retrieving relevant documents from an external knowledge base and incorporating them into the input.

Advantages:

- Improves factual accuracy
- Reduces hallucination
- Enables access to external knowledge

Limitations:

- Retrieval often includes redundant or irrelevant information
- Lack of proper ranking and filtering
- No strict control over token usage

2.2.3 Semantic Retrieval Systems

Semantic retrieval uses dense vector representations generated by embedding models such as Sentence-BERT [3]. These systems capture contextual meaning and retrieve semantically similar documents.

Advantages:

- Effective for conceptual queries
- Captures contextual similarity

Limitations:

- May fail to retrieve exact keyword matches
- Sensitive to embedding quality

2.2.4 Lexical Retrieval (BM25)

BM25 is a probabilistic retrieval algorithm that ranks documents based on keyword frequency and relevance [4].

Advantages:

- Accurate keyword matching
- Effective for named entities and technical terms

Limitations:

- Does not capture semantic meaning
- Ineffective for paraphrased queries

2.2.5 Hybrid Retrieval Approaches

Hybrid retrieval combines semantic and lexical search methods to improve retrieval performance. Techniques such as Reciprocal Rank Fusion (RRF) [5] are used to merge results from multiple retrieval systems.

This approach improves both recall and precision but requires careful ranking and filtering mechanisms.

2.3 Related Works

Several research works have analyzed the limitations of LLMs and proposed improvements:

- Vaswani et al. [1] introduced the Transformer architecture, enabling parallel sequence processing using self-attention.
- Lewis et al. [2] proposed Retrieval-Augmented Generation, integrating external knowledge into language models.
- Reimers and Gurevych [3] developed Sentence-BERT, enabling efficient semantic embedding generation.
- Johnson et al. [6] introduced FAISS, a scalable vector search library for high-dimensional similarity search.

- Liu et al. [7] demonstrated the “lost-in-the-middle” effect, showing that LLMs fail to retrieve centrally located information.
- Gao et al. [8] provided a comprehensive survey of RAG systems, highlighting the importance of chunking and retrieval quality.
- Recent studies such as *Long Context vs RAG* and *Context Rot* show that increasing input length introduces noise and reduces model performance.
- Research on intelligence degradation in long-context models identifies thresholds beyond which accuracy declines significantly.

2.4 Tools / Technologies Used

2.4.1 FAISS (Vector Database) FAISS is used for efficient similarity search in high-dimensional vector spaces [6]. It enables fast retrieval of semantically similar document chunks using dense embeddings.

2.4.2 BM25 (Lexical Retrieval Algorithm) BM25 is used for keyword-based document retrieval. It ensures accurate matching of exact terms, especially for technical queries.

2.4.3 Sentence-BERT (Embedding Model) Sentence-BERT is used to generate dense vector representations of text, enabling semantic similarity search.

2.4.4 Cross-Encoder Reranker A cross-encoder model is used to rerank retrieved documents by jointly evaluating the query and document, improving retrieval accuracy.

2.4.5 Caching Mechanisms Query-level and response-level caching are used to reduce redundant computations and improve system efficiency.

2.6 Research Gap

The literature reveals several unresolved issues:

- Increasing context window size leads to higher computational cost without solving attention-related problems
- Standard RAG systems lack effective filtering and ranking mechanisms

- Semantic retrieval alone fails for keyword-specific queries
- Lexical retrieval lacks contextual understanding
- Existing systems do not integrate hybrid retrieval, reranking, and caching in a unified architecture
- Token budget control is often not enforced
- Limited real-world implementations exist with persistent external memory

These gaps highlight the need for a hybrid retrieval-based system that efficiently selects relevant context while minimizing computational overhead.

2.5 Comparative Analysis of Existing Approaches

Approach	Strengths	Limitations
Transformer (Long Context)	Strong contextual understanding	High cost, attention dilution
RAG	Improves factual grounding	Poor filtering, high token usage
Semantic Retrieval	Captures meaning	Misses exact keywords
Lexical Retrieval (BM25)	Accurate keyword matching	No semantic understanding
Hybrid Retrieval	Combines strengths	Requires ranking and optimization

This comparison shows that no single approach fully addresses all challenges associated with long-context processing.

2.7 Summary

This chapter presented a review of existing approaches and technologies related to large language models and retrieval systems. While significant progress has been made, limitations in long-context processing, retrieval quality, and efficiency remain unresolved.

The identified research gap motivates the design of a hybrid retrieval-based architecture with external memory, which is discussed in the next chapter.

CHAPTER 3
DESIGN OF THE PROPOSED SYSTEM

3.1 System Architecture

The proposed system is a modular Retrieval-Augmented Generation (RAG) architecture developed to mitigate context degradation in Large Language Models by separating long-document storage from query-time inference. Instead of providing entire documents as input to the model, the system retrieves only the most relevant content and enforces a strict token budget. The architecture is organized into five layers:

1. **Ingestion Layer**
   Documents in formats such as PDF and text are ingested and converted into normalized plain text. Noise and formatting artifacts are removed to ensure consistency.
2. **Representation Layer**
   The normalized text is divided into overlapping chunks of approximately 500 tokens with a 50-token overlap. This overlap preserves contextual continuity across chunk boundaries. Each chunk is converted into a dense vector using a sentence-transformers embedding model.
3. **Indexing and Storage Layer**
   Two parallel indexes are constructed:
   • A FAISS vector index for semantic similarity search
   • A BM25 inverted index (rank_bm25) for lexical keyword retrieval

Both indexes are built during ingestion and persisted for efficient reuse.

4. **Hybrid Retrieval Layer**
   At query time, both FAISS and BM25 searches are executed in parallel. Their ranked outputs are merged using Reciprocal Rank Fusion (RRF) with a smoothing parameter k = 60. The fused candidate set is then refined using a cross-encoder (BERT-based) reranker, which jointly evaluates the query and each candidate passage.
5. **Synthesis Layer**
   The top-ranked chunks are selected under a strict token budget of 1000–2000 tokens.

These chunks are concatenated with the user query to form a context-aware prompt, which is sent to the Large Language Model. A dual-layer caching mechanism is

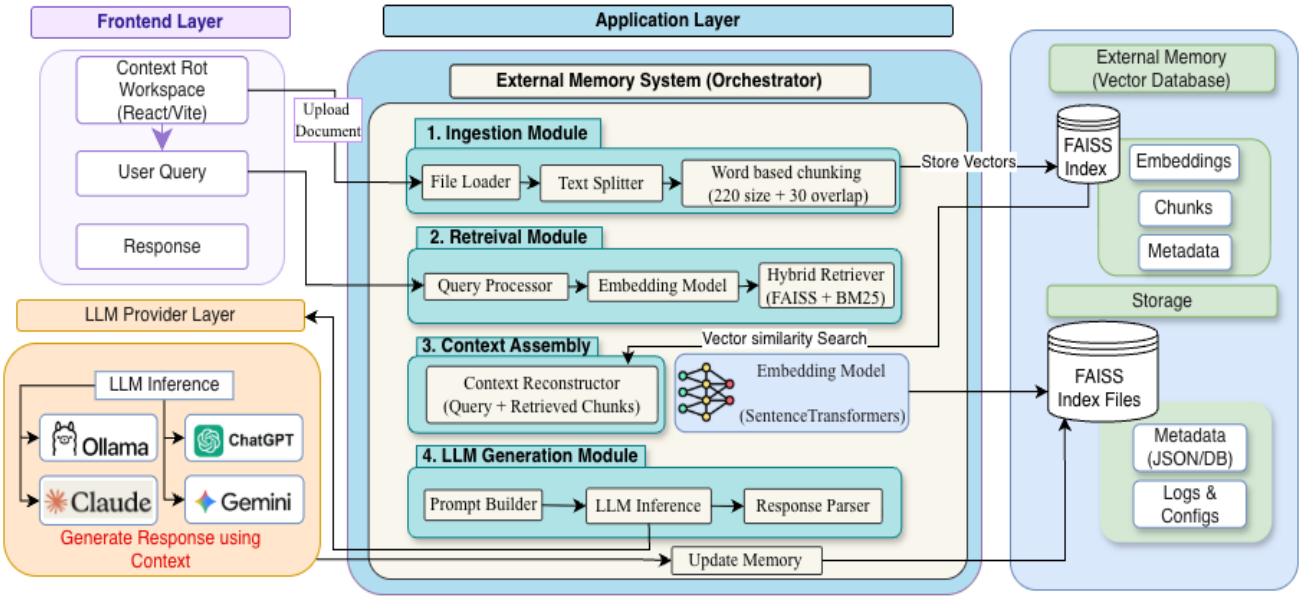

Figure 3.1: Overall Architecture of the Proposed Hybrid Retrieval System

Description:

The figure illustrates the end-to-end architecture of the system. Documents are ingested, processed into chunks, and converted into embeddings. These embeddings are stored in a FAISS index, while BM25 indexes the textual content. During query processing, both retrieval systems operate in parallel. Their outputs are combined using Reciprocal Rank Fusion and further refined using a cross-encoder reranker. The selected context is then passed to the language model under a strict token constraint, with caching applied to improve performance.

3.2 System Requirements – Software and Hardware

This section outlines the software and hardware requirements necessary for implementing the proposed system.

3.2.1 Software Requirements

• Programming Language: Python

• Libraries: FAISS, sentence-transformers, rank_bm25

• Reranking Model: Cross-encoder (BERT-based)

• LLM APIs: Gemini, GPT, or Claude

• Browser Extension: JavaScript (MutationObserver)

• Operating System: Windows, Linux, or macOS

3.2.2 Hardware Requirements

• Minimum 8 GB RAM (16 GB recommended)

• Multi-core CPU

• Optional GPU for faster embedding and reranking operations

• Stable internet connection for LLM API access

3.3 Data Flow Diagrams (DFD)

The data flow of the proposed system is represented using a Data Flow Diagram, illustrating the sequence of operations during document indexing and query processing.

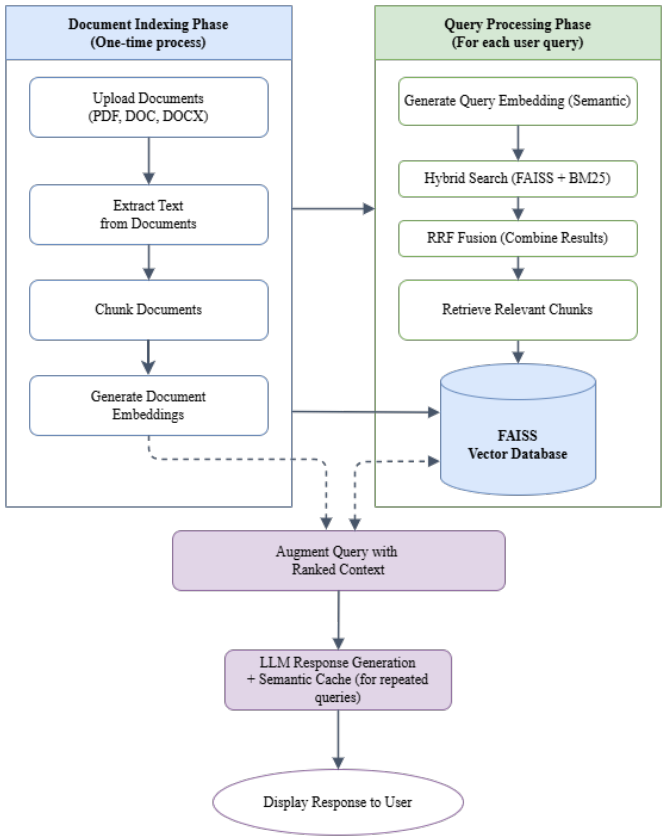

Figure 3.2: Information Retrieval Workflow

Description:

The workflow is divided into two primary phases:

(A) Document Indexing Phase

• Documents are uploaded and converted into text
• The text is segmented into overlapping chunks (500 tokens with 50-token overlap)
• Each chunk is transformed into a dense embedding
• Embeddings are stored in the FAISS index
• A BM25 inverted index is built from the same chunk set

This phase is executed once and enables efficient retrieval during subsequent queries.

(B) Query Processing Phase

• The user query is converted into an embedding
• FAISS (semantic) and BM25 (lexical) searches are executed in parallel
• Results are merged using Reciprocal Rank Fusion (k = 60)
• A cross-encoder reranker refines the fused candidates
• Top-ranked chunks are selected under the token budget
• A context-aware prompt is constructed
• The Large Language Model generates the final response

Caching Mechanism

• **Query Cache:** Stores recent query embeddings and retrieved contexts using an LRU strategy with a time-to-live policy. Similar queries are identified using cosine similarity and served from cache when applicable.
• **Response Cache:** Returns previously generated responses for semantically similar queries, reducing repeated calls to the language model.

This workflow ensures efficient retrieval of relevant information while minimizing token usage and computational overhead.

3.4 Use Case Diagram

The system involves the following actors and their interactions:

Actors:

• User
• Browser Extension
• Backend Retrieval System
• Large Language Model

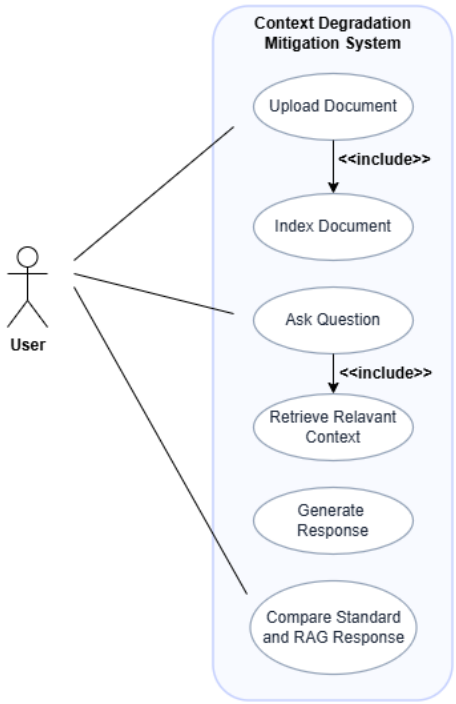

Figure 3.3: Use Case Diagram of the Proposed Context Degradation Mitigation System

Description:

The figure illustrates the use case diagram of the proposed system. The primary actor is the user, who interacts with the system to upload documents, submit queries, and receive responses. The system internally performs document indexing, retrieval of relevant context, and response generation using a Large Language Model. The inclusion relationships indicate that document indexing is part of the upload process, while context retrieval is part of query processing. The

system also supports comparison between standard language model responses and retrieval-augmented responses.

Use Cases:

• Upload and process documents
• Generate embeddings and build indexes
• Submit user queries
• Retrieve relevant document chunks
• Rerank and construct context
• Generate responses using the language model
• Cache query and response results
• Inject conversational memory across platforms

3.5 Cross-Platform Conversational Memory Module

Since the proposed system relies on vector-based indexing and retrieval rather than a traditional relational database, an Entity-Relationship (ER) diagram is not applicable. Instead, the system incorporates a cross-platform conversational memory module, described below.

A browser extension is integrated as an application layer over the retrieval system to enable persistent conversational memory across multiple platforms.

The extension uses a MutationObserver to monitor changes in the Document Object Model (DOM) on platforms such as ChatGPT, Gemini, and Claude. It extracts completed prompt-response pairs and sends them to the backend system for processing.

These interactions are:
• Chunked and embedded
• Stored in a FAISS-based conversational index
• Indexed using BM25 for keyword-based retrieval

When a new query is submitted:
• The extension retrieves relevant past interactions

• Injects them into the input context
• Enhances continuity across sessions and platforms

This module improves user experience by enabling persistent, context-aware interactions across different AI platforms.

CHAPTER 4

IMPLEMENTATION OF THE PROPOSED SYSTEM

4.1 Technologies Used

The proposed system is implemented using a combination of modern programming frameworks, machine learning libraries, and web technologies to ensure efficient and scalable performance.

The backend is developed using the Python programming language, with FastAPI employed as the framework for building RESTful APIs. FastAPI enables efficient handling of user queries and document ingestion through well-defined endpoints.

Several specialized libraries are used to support different components of the system. FAISS is utilized for efficient vector similarity search, enabling semantic retrieval of relevant document chunks. The sentence-transformers library is used to generate dense embeddings from textual data. The rank_bm25 library is employed for lexical retrieval based on keyword matching. NumPy is used for numerical operations, and the requests library facilitates communication with external language model APIs.

The system integrates multiple Large Language Models, including Gemini, GPT, and Claude, through REST API interfaces. These models are used to generate responses based on the constructed context.

A browser extension is implemented using vanilla JavaScript. The MutationObserver API is used to monitor changes in the Document Object Model, allowing real-time extraction of conversational data. The extension communicates with the backend using HTTP requests.

The implementation is platform-independent and can be deployed on Windows, Linux, and macOS systems.

4.2 System Modules and Description

The system follows a modular architecture, where each module is responsible for a specific stage of processing.

The document ingestion module processes input documents and converts them into normalized textual format. The preprocessing and chunking module segments the text into overlapping chunks using a token-based sliding window approach, ensuring contextual continuity.

The embedding module converts each chunk into a dense vector representation using the all-MiniLM-L6-v2 model. The indexing module builds both FAISS and BM25 indexes, enabling efficient retrieval.

The retrieval module performs parallel semantic and lexical searches. The fusion module combines these results using Reciprocal Rank Fusion. The reranking module refines the results using a cross-encoder model.

The context assembly module selects the most relevant chunks and constructs the final context within a specified token limit. The language model interaction module sends this context to the LLM and retrieves the generated response.

The caching module stores previously processed queries and responses to improve performance. The browser extension module enables cross-platform conversational memory by capturing and reusing past interactions.

4.3 Algorithm and Methodology

The implementation integrates multiple algorithms to achieve efficient and accurate retrieval.

The input text is segmented using a sliding window approach with a chunk size of 500 tokens and an overlap of 50 tokens. This ensures that contextual information is preserved across chunk boundaries.

Each chunk is converted into a dense vector representation using the all-MiniLM-L6-v2 embedding model, which produces embeddings of 384 dimensions.

For semantic retrieval, the FAISS index is used with cosine similarity as the distance metric. The system retrieves the top 10 nearest neighbor document chunks for each query.

For lexical retrieval, the BM25 algorithm is applied using tokenized and lowercased text. The system retrieves the top 10 documents based on keyword relevance.

The outputs from both retrieval methods are combined using Reciprocal Rank Fusion. In this implementation, the RRF score is computed by summing the reciprocal of the rank of each document across both retrieval lists. The score is calculated as one divided by the sum of the constant k and the document rank, where k is set to 60. This approach ensures that documents ranked highly in either retrieval method contribute significantly to the final ranking.

The top 20 fused candidates are then passed to a cross-encoder model based on the MS MARCO architecture. The model evaluates each query-document pair and assigns a relevance score, which is used to reorder the candidates.

After reranking, the top 5 highest-scoring document chunks are selected. These chunks are sequentially added to the context until the token limit of approximately 1000 to 2000 tokens is reached. The chunks are selected in descending order of relevance score.

4.4 Implementation Details

The implementation follows a sequential processing pipeline in which each module contributes to transforming the user query into a context-aware response.

When a user submits a query, the system first converts the query into a vector embedding. Semantic and lexical retrieval processes are then executed in parallel using FAISS and BM25 respectively. Each method retrieves the top 10 candidate document chunks.

The retrieved results are combined using Reciprocal Rank Fusion, producing a unified list of ranked candidates. The top 20 candidates are then passed to a cross-encoder model for reranking. Based on the reranker scores, the top 5 most relevant chunks are selected.

These selected chunks are combined with the user query to construct a context-aware prompt. The system ensures that the total token count remains within the predefined limit by stopping the addition of further chunks once the threshold is reached.

The constructed prompt is sent to the language model through a REST API call, and the generated response is returned to the user in JSON format.

To improve performance, a dual-layer caching mechanism is implemented. The query cache stores embeddings and retrieval results using a least-recently-used strategy. Cosine similarity with a threshold of 0.85 is used to identify similar queries. The response cache stores previously generated outputs and returns them for semantically similar queries, reducing redundant API calls.

The browser extension continuously monitors the web interface using the MutationObserver API. It captures complete prompt-response pairs and sends them to the backend system. These interactions are stored and indexed, enabling retrieval of relevant past interactions for future queries.

The backend exposes RESTful endpoints such as /query and /upload to facilitate communication between the frontend and the retrieval system.

Basic error handling mechanisms are implemented to manage cases such as empty queries and scenarios where retrieval results are unavailable. In such cases, the system falls back to the available retrieval method or returns an appropriate response.

4.5 Optimization and Limitations

The system incorporates several optimization techniques to improve performance. Hybrid retrieval combines semantic and lexical search methods to enhance accuracy. The dual-layer caching mechanism reduces latency by avoiding redundant computations. Token budgeting ensures efficient utilization of the language model’s context window.

Despite these optimizations, certain limitations remain. The cross-encoder reranker introduces additional computational overhead, which increases response time. The overall system latency is

approximately two seconds, with the majority of the delay attributed to external language model API calls. Additionally, system performance is dependent on the quality of embeddings and the effectiveness of the chunking strategy.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Experimental Setup

The proposed system was evaluated using a collection of approximately twenty documents in PDF and DOCX formats. These documents ranged in size from 50 to 200 pages, representing realistic long-context scenarios.

The evaluation included both technical and conceptual queries. Technical queries focused on keyword-heavy inputs and named entities, while conceptual queries required semantic understanding and contextual reasoning. The distribution of queries was balanced, with approximately fifty percent technical and fifty percent conceptual queries.

A baseline approach was established by providing the entire document directly as input to the language model without any retrieval mechanism. The performance of this baseline was compared with the proposed hybrid retrieval system.

The evaluation methodology involved applying the same set of queries to both systems. The comparison was conducted using estimated token usage and qualitative assessment of response relevance and consistency.

5.2 Performance Analysis

The system was evaluated based on token efficiency, response latency, and output consistency.

Table 5.1: System Performance Comparison

Metric	**Baseline (Full Context)**	**Hybrid Retrieval System**
Average Token Usage	Approximately 10,000 or more tokens	Approximately 1,000 to 2,000 tokens
Inference Latency	High	Significantly reduced
Response Stability	Variable	Consistent and stable

The token usage values are estimated based on strict chunk size limits and retrieval constraints. The baseline approach processes the entire document, resulting in significantly higher token usage. In contrast, the proposed system limits the context to only the most relevant chunks, leading to substantial reduction in token consumption.

Latency measurements are observational. The proposed system demonstrates visibly reduced response time, primarily due to efficient retrieval and the use of caching mechanisms.

The system also exhibits improved response stability, producing more consistent outputs with reduced hallucination compared to the baseline approach.

5.3 Observations and Findings

The experimental evaluation highlights several important findings.

The integration of BM25 significantly improves the retrieval of exact technical phrases. For example, queries involving terms such as “positional encoding” are accurately matched using lexical retrieval, which may not be effectively captured by semantic retrieval alone.

Semantic retrieval using FAISS effectively captures contextual meaning but may fail to retrieve exact keywords when they are embedded within dense conceptual text. This limitation is addressed by combining semantic and lexical retrieval methods.

The hybrid retrieval approach successfully balances these strengths, resulting in improved overall retrieval accuracy.

The use of Reciprocal Rank Fusion enables effective merging of results from both retrieval methods. However, since the fusion parameter remains fixed, highly ambiguous queries may occasionally result in mixed or less precise document selection.

The cross-encoder reranker plays a critical role in improving relevance. It effectively filters out loosely related chunks and refines the ranking of candidate documents, leading to more accurate context selection.

Despite these improvements, minor limitations were observed. In certain cases, especially with highly ambiguous queries, the system may include slightly irrelevant chunks due to the static nature of the fusion weights.

5.4 Screenshots / Sample Outputs

The evaluation of the proposed system is supported through analysis of system behavior and performance metrics rather than user interface screenshots.

No direct screenshots of query outputs or browser extension interactions are included. Instead, the results are demonstrated using architectural diagrams, workflow representations, and comparative analysis presented in earlier chapters.

The system consistently retrieves relevant document chunks and generates context-aware responses based on the hybrid retrieval process. The browser extension component successfully captures and utilizes past interactions, enabling improved contextual continuity across sessions.

These observations confirm that the system operates as intended and effectively mitigates context degradation in large language models.

CHAPTER 6

CONCLUSIONS AND FUTURE WORK

6.1 Conclusions

This work presents a hybrid retrieval-based approach to address the limitations of large language models in long-context processing. The proposed system integrates semantic retrieval using FAISS with lexical retrieval using BM25, combined through Reciprocal Rank Fusion. This hybrid strategy ensures both contextual relevance and precise keyword matching.

The incorporation of a cross-encoder reranking mechanism further refines the retrieved results by evaluating query-document pairs jointly, improving the overall quality of the selected context. In addition, the system enforces strict token budget control, limiting the input to the most relevant information and thereby improving efficiency.

The experimental evaluation demonstrates that the proposed approach significantly improves token efficiency, reduces response latency, and enhances response stability. By mitigating issues such as attention dilution and the lost-in-the-middle effect, the system delivers more accurate and consistent outputs compared to traditional full-context approaches.

Overall, the integration of hybrid retrieval, fusion techniques, reranking, and controlled context generation provides an effective solution for improving the performance of large language models in real-world applications.

6.1.1 Limitations

Despite the improvements achieved, certain limitations remain.

The use of a static smoothing parameter in Reciprocal Rank Fusion limits the adaptability of the system across different datasets and query types. In addition, the cross-encoder reranking process introduces computational overhead, which may slightly increase response time.

The system may also produce less precise results for highly ambiguous queries, as the fixed weighting mechanism in the fusion stage does not dynamically adjust to varying query characteristics.

6.2 Recommendations / Future Work

Future work can focus on enhancing the adaptability and efficiency of the system.

One potential improvement is the use of adaptive weighting mechanisms in the Reciprocal Rank Fusion stage, allowing the system to dynamically adjust the importance of semantic and lexical retrieval based on the query.

Dynamic chunking strategies can also be explored, where chunk size and overlap are adjusted based on document structure and complexity, improving context preservation.

Incorporating domain-specific fine-tuning of embedding and reranking models can further enhance retrieval accuracy for specialized datasets.

Additionally, adaptive token budgeting techniques can be developed to optimize context selection dynamically, ensuring efficient utilization of the model's input capacity.

These enhancements have the potential to further improve the robustness and scalability of retrieval-augmented language model systems.

REFERENCES

[1] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

[3] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[4] X. Yang et al., “GPT: A Comprehensive Review on Enabling Technologies, Potential Applications, Emerging Challenges, and Future Directions,” 2023.

[5] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[6] Z. Gao et al., “Retrieval-Augmented Generation for Large Language Models: A Survey,” 2024.

[7] J. Johnson et al., “FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors,” 2019.

[8] A. Xie et al., “Long Context vs. RAG for LLMs: An Evaluation and Revisit,” 2024.

[9] Y. Li et al., “Intelligence Degradation in Long-Context LLMs: Critical Threshold Determination via Natural Length Distribution Analysis,” 2024.

[10] “Context Rot: How Increasing Input Tokens Impacts LLM Performance,” 2024.

[11] S. E. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, 2009.

[12] G. V. Cormack et al., “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” *Proceedings of SIGIR*, 2009.